

Security Monitoring and Detection Infrastructure

Saad Korchi

Rapport SIEM — Architecture SOC, Déploiement & Analyse

Matière : Sécurité des Réseaux / Infrastructure SOC

Stack : ELK Stack • Suricata • Filebeat • DVWA • Zabbix

Environnement : 3 VMs — Ubuntu Server ×2, Kali Linux

Rapport réalisé dans un cadre académique et professionnel. Tous les tests ont été effectués dans un environnement de laboratoire virtuel isolé.

Table des matières

1	Introduction au SIEM	4
1.1	Définition et rôle d'un SIEM	4
1.2	Objectifs du projet	4
1.3	Méthodologie	4
2	Architecture Mise en Place	4
2.1	Vue d'ensemble	4
2.2	Inventaire des machines virtuelles	4
2.3	Topologie réseau	5
2.4	Flux de données	5
2.5	Configuration du routage (étape critique)	5
2.5.1	Activation du forwarding IP sur le SIEM	5
2.5.2	Redirection de la passerelle sur WEB	6
2.5.3	Redirection de la passerelle sur KALI	6
3	Description des Agents	6
3.1	Vue d'ensemble des agents déployés	6
3.2	Filebeat	6
3.2.1	Architecture modulaire de Filebeat	6
3.2.2	Configuration Filebeat sur WEB (Apache)	7
3.2.3	Configuration Filebeat sur SIEM (Suricata)	7
3.3	Suricata	7
3.3.1	Installation et mise à jour des règles	7
3.3.2	Vérification du fonctionnement	7
3.4	Zabbix Agent	8
4	Collecte et Corrélation des Logs	8
4.1	Sources de logs	8
4.2	Structure du log Apache (access log)	8
4.3	Structure d'une alerte Suricata (eve.json)	8
4.4	Corrélation dans Kibana Discover	9
4.5	Problèmes rencontrés lors de la collecte	9
4.5.1	Filebeat — aucun événement dans Kibana	9
4.5.2	Elasticsearch — statut RED au démarrage	9
4.5.3	Kibana — inaccessible depuis le réseau	10
5	Scénarios de Monitoring et Création des Alertes	10
5.1	Scénarios implémentés	10
5.1.1	Scénario 1 — Injection SQL via SQLMap	10
5.1.2	Scénario 2 — Injection SQL via l'interface DVWA (navigateur)	11
5.2	Règles Suricata déclenchées	11
5.3	Configuration des alertes Suricata	11
6	Analyse d'Incidents	11
6.1	Méthodologie d'analyse	11
6.2	Incident 1 — Attaque par injection SQL (SQLMap)	12
6.3	Incident 2 — Problème de configuration (faux incident)	12

7	Dashboards Kibana / Zabbix	13
7.1	Kibana — Vue Discover	13
7.1.1	Index pattern	13
7.1.2	Filtres utilisés pour la corrélation	13
7.1.3	Dashboards Kibana pré-intégrés via Filebeat	13
7.2	Visualisations personnalisées recommandées	14
7.3	Zabbix — Monitoring Infrastructure	14
8	Logs Collectés	15
8.1	Récapitulatif des logs par source	15
8.2	Exemple de log Apache annoté	15
8.3	Commandes de consultation des logs	15
9	Leçons Apprises	16
10	Conclusion	16

1 Introduction au SIEM

1.1 Définition et rôle d'un SIEM

Un **SIEM (Security Information and Event Management)** est une solution centralisée qui collecte, normalise, corrèle et analyse les événements de sécurité provenant de l'ensemble du système d'information en temps réel. Il constitue le cœur opérationnel d'un SOC (Security Operations Centre).

Les deux fonctions fondamentales d'un SIEM sont :

- **SIM (Security Information Management)** — archivage long terme des logs, conformité réglementaire, recherches forensiques.
- **SEM (Security Event Management)** — corrélation en temps réel, détection d'anomalies, déclenchement d'alertes.

1.2 Objectifs du projet

Ce projet a pour objectif de concevoir, déployer et valider une infrastructure SOC fonctionnelle reproduisant les mécanismes d'un environnement d'entreprise réel :

1. Mettre en place une architecture réseau maîtrisée avec visibilité totale du trafic.
2. Déployer un stack de détection complet (Suricata + ELK + Filebeat).
3. Générer des attaques réelles depuis une machine Kali Linux.
4. Collecter et corrélérer les logs applicatifs et réseau dans Kibana.
5. Surveiller l'infrastructure système avec Zabbix.

1.3 Méthodologie

La démarche suivie est itérative et orientée incident : chaque composant est déployé indépendamment, testé, puis intégré à l'architecture globale. Chaque erreur rencontrée fait l'objet d'une analyse de cause racine, d'une correction documentée et d'une validation.

2 Architecture Mise en Place

2.1 Vue d'ensemble

L'architecture repose sur trois machines virtuelles interconnectées sur un réseau privé 192.168.1.0/24. Le nœud SIEM occupe une position stratégique de passerelle obligatoire : tout le trafic entre la machine attaquante et la cible transite par lui, garantissant une visibilité totale à l'IDS.

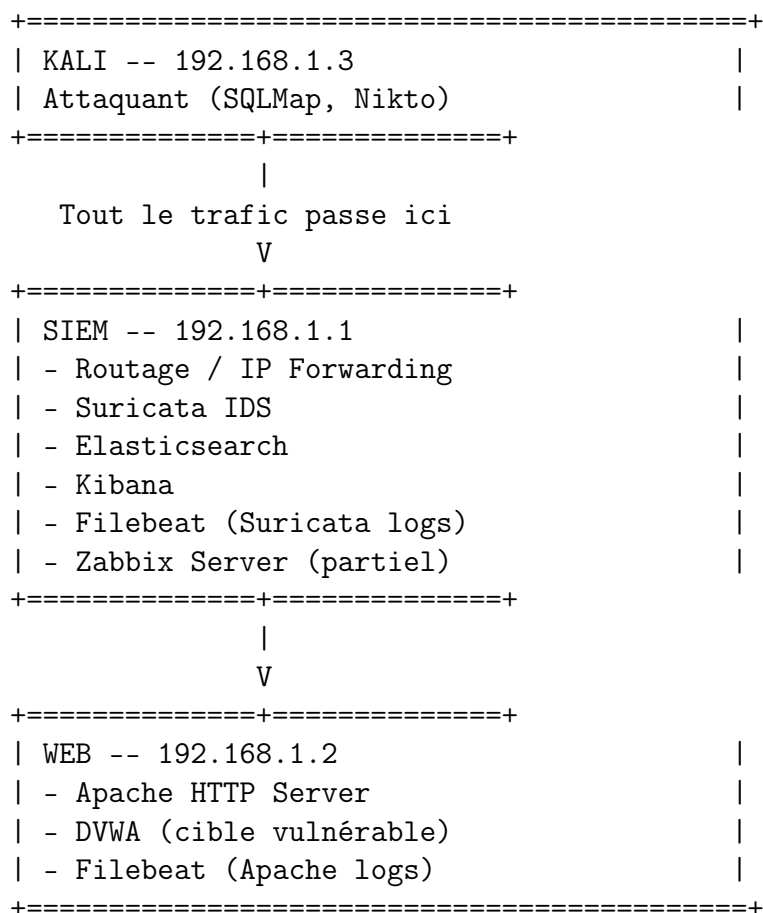
2.2 Inventaire des machines virtuelles

VM	Système d'exploitation	Adresse IP	Rôle
SIEM	Ubuntu Server (TTY)	192.168.1.1	Routage, IDS, ELK, Zabbix
WEB	Ubuntu Server	192.168.1.2	Apache + DVWA + Filebeat
KALI	Kali Linux	192.168.1.3	Machine attaquante

TABLE 1 – Inventaire des machines virtuelles

2.3 Topologie réseau

Le plan d'adressage statique a été retenu pour éviter tout aléa de routage dynamique. La passerelle de chaque VM est configurée à 192.168.1.1 (le SIEM), ce qui force tout le trafic à le traverser.



2.4 Flux de données

Source	Destination	Type de données
KALI	WEB (via SIEM)	Requêtes HTTP d'attaque
WEB	SIEM (Filebeat)	Logs Apache (access + error)
SIEM	Elasticsearch	Alertes Suricata (eve.json)
Elasticsearch	Kibana	Indexation et visualisation

TABLE 2 – Flux de données dans l'architecture

2.5 Configuration du routage (étape critique)

Sans ce routage, Suricata ne voit aucun trafic car les paquets circulent directement entre KALI et WEB en contournant le SIEM.

2.5.1 Activation du forwarding IP sur le SIEM

```
1 sudo sysctl -w net.ipv4.ip_forward=1
```

Listing 1 – Activation du forwarding IP — temporaire

```
1 echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
```

Listing 2 – Persistance du forwarding IP au redémarrage

2.5.2 Redirection de la passerelle sur WEB

```
1 sudo ip route replace default via 192.168.1.1
```

Listing 3 – Forcer WEB à passer par le SIEM

2.5.3 Redirection de la passerelle sur KALI

```
1 sudo ip route replace default via 192.168.1.1
```

Listing 4 – Forcer KALI à passer par le SIEM

Résultat : Le trafic KALI WEB transite désormais intégralement par le SIEM. Suricata inspecte chaque paquet. Cette étape est la condition préalable indispensable à toute détection réseau.

3 Description des Agents

3.1 Vue d'ensemble des agents déployés

Trois types d'agents/services assurent la collecte, la détection et l'indexation :

Agent	VM	Rôle	Source de données
Filebeat	WEB	Shipper de logs	Logs Apache (/var/log/apache2/)
Filebeat	SIEM	Shipper de logs	Alertes Suricata (/var/log/suricata/eve.json)
Suricata	SIEM	IDS réseau	Trafic réseau en temps réel
Zabbix Agent	Toutes VMs	Monitoring système	CPU, RAM, disque, interfaces

TABLE 3 – Agents déployés

3.2 Filebeat

Filebeat est un shipper léger développé par Elastic. Il surveille des fichiers de log en continu, applique des parsers structurés (modules), et envoie les événements vers Elasticsearch ou Logstash.

3.2.1 Architecture modulaire de Filebeat

Filebeat fonctionne sur un modèle `module / fileset` :

- Un **module** regroupe la configuration pour un produit donné (apache, suricata, nginx...).

— Un **fileset** est une sous-configuration d'un module ciblant un type de log spécifique. Les deux commandes suivantes sont toutes deux obligatoires :

```
1 sudo filebeat modules enable apache    # active le module
2 sudo filebeat setup                    # enregistre les pipelines et
   dashboards
3 sudo systemctl restart filebeat
```

Listing 5 – Activation d'un module Filebeat (exemple Apache)

3.2.2 Configuration Filebeat sur WEB (Apache)

Le fichier `/etc/filebeat/modules.d/apache.yml` déclare les chemins des logs :

```
1 - module: apache
2   access:
3     enabled: true
4     var.paths: ["/var/log/apache2/access.log*"]
5   error:
6     enabled: true
7     var.paths: ["/var/log/apache2/error.log*"]
```

Listing 6 – Module Apache — filebeat (WEB VM)

3.2.3 Configuration Filebeat sur SIEM (Suricata)

```
1 - module: suricata
2   eve:
3     enabled: true
4     var.paths: ["/var/log/suricata/eve.json"]
```

Listing 7 – Module Suricata — filebeat (SIEM VM)

3.3 Suricata

Suricata est un IDS/IPS open source haute performance. Il analyse le trafic réseau en temps réel à partir d'une interface réseau, applique un jeu de règles de signature, et génère des alertes structurées au format JSON (eve.json).

3.3.1 Installation et mise à jour des règles

```
1 sudo apt install suricata -y
2 sudo suricata-update          # telecharge les regles Emerging Threats
3 sudo systemctl enable suricata
4 sudo systemctl start suricata
```

Listing 8 – Installation de Suricata et mise à jour des règles

3.3.2 Vérification du fonctionnement

```
1 sudo systemctl status suricata
2 sudo tail -f /var/log/suricata/eve.json
```

Listing 9 – Vérification du service et des logs live

3.4 Zabbix Agent

Zabbix surveille les métriques système (CPU, mémoire, disque, interfaces réseau) sur chaque VM et remonte les données vers le Zabbix Server hébergé sur le SIEM.

```
1 sudo apt install zabbix-server-mysql zabbix-frontend-php \  
2 zabbix-apache-conf zabbix-sql-scripts zabbix-agent -y
```

Listing 10 – Tentative d'installation de Zabbix Server

Erreur rencontrée : `unable to correct problems, you have held broken packages`
Cause : Le dépôt Zabbix ajouté ne correspondait pas à la version Ubuntu en cours, créant une chaîne de dépendances cassée. Ce problème est indépendant du pipeline de détection SIEM. La résolution nécessite d'identifier la version Ubuntu exacte avec `lsb_release -a` et d'ajouter le dépôt Zabbix correspondant.

4 Collecte et Corrélation des Logs

4.1 Sources de logs

Dans cette architecture, deux flux de logs principaux alimentent Elasticsearch :

- **Logs Apache** (`event.module: apache`) — générés par le serveur web sur la VM WEB et expédiés par Filebeat.
- **Alertes Suricata** (`event.module: suricata`) — générées par l'IDS sur la VM SIEM et expédiées par le second Filebeat.

4.2 Structure du log Apache (access log)

Un log d'accès Apache typique suit le format Combined Log Format :

```
1 192.168.1.3 - - [02/Jun/2026:14:32:10 +0000] "GET /vulnerabilities/sqli/  
2 ?id=1%27+AND+1%3D1--&Submit=Submit HTTP/1.1" 200 4823 "http  
   ://192.168.1.2/vulnerabilities/sqli/" "sqlmap/1.7 (..."
```

Listing 11 – Exemple d'entrée dans access.log lors d'une attaque SQLMap

4.3 Structure d'une alerte Suricata (eve.json)

Le fichier `eve.json` stocke les événements Suricata au format JSON ligne par ligne :

```
1 {  
2   "timestamp": "2026-06-02T14:32:10.123456+0000",  
3   "event_type": "alert",  
4   "src_ip": "192.168.1.3",  
5   "src_port": 54321,  
6   "dest_ip": "192.168.1.2",  
7   "dest_port": 80,  
8   "proto": "TCP",  
9   "alert": {  
10    "action": "allowed",  
11    "gid": 1,  
12    "signature_id": 2006445,  
13    "rev": 10,  
14    "signature": "ET WEB_SERVER SQL Injection Attempt",
```



```

15     "category": "Web Application Attack",
16     "severity": 1
17 },
18 "http": {
19     "hostname": "192.168.1.2",
20     "url": "/vulnerabilities/sqlmap/?id=1%27+AND+1%3D1--+",
21     "http_user_agent": "sqlmap/1.7"
22 }
23 }

```

Listing 12 – Exemple d’alerte Suricata

4.4 Corrélation dans Kibana Discover

Une fois les deux modules Filebeat actifs, les événements des deux sources apparaissent dans l’index `filebeat-*` de Kibana Discover. La corrélation s’effectue par :

Champ de corrélation	Valeur exemple	Signification
source.ip / src_ip	192.168.1.3	Machine attaquante
@timestamp	2026-06-02T14 :32 :10	Synchronisation temporelle
url.original	?id=1%27+AND...	Payload SQL dans les deux logs
event.module	apache / suricata	Source du log

TABLE 4 – Champs de corrélation

Résultat de la corrélation : En filtrant sur `source.ip`: 192.168.1.3 dans Kibana Discover, l’analyste voit simultanément les requêtes HTTP (Apache) et les alertes IDS (Suricata) pour la même attaque, avec un alignement temporel précis permettant de reconstruire la chaîne d’attaque complète.

4.5 Problèmes rencontrés lors de la collecte

4.5.1 Filebeat — aucun événement dans Kibana

Erreur : `apache is configured but no enabled filesets`

Cause : Le module Apache était activé dans la configuration mais les filesets n’avaient pas été initialisés avec `filebeat setup`.

```

1 sudo filebeat modules enable apache
2 sudo filebeat setup
3 sudo systemctl restart filebeat

```

Listing 13 – Correction — activation complète du module Apache

4.5.2 Elasticsearch — statut RED au démarrage

Problème : L’API retournait `"status": "red"` — shards non assignés, index système non initialisés.

```

1 curl http://localhost:9200/_cluster/health?pretty
2 sudo systemctl restart elasticsearch # Attendre ~60s puis rev rifier
3 curl http://localhost:9200/_cluster/health?pretty

```

Listing 14 – Diagnostic et redémarrage Elasticsearch

4.5.3 Kibana — inaccessible depuis le réseau

Problème : Kibana démarré mais non accessible sur 192.168.1.1:5601.

Cause : Kibana lié à localhost uniquement (127.0.0.1).

```
1 sudo nano /etc/kibana/kibana.yml
2 # Modifier :
3 # server.host: "localhost"
4 # En :
5 # server.host: "0.0.0.0"
6 sudo systemctl restart kibana
```

Listing 15 – Correction — liaison sur toutes les interfaces

5 Scénarios de Monitoring et Création des Alertes

5.1 Scénarios implémentés

5.1.1 Scénario 1 — Injection SQL via SQLMap

Objectif : Détecter une attaque par injection SQL automatisée.

Contexte : DVWA est déployé sur la VM WEB avec un niveau de sécurité Low. L'attaquant (KALI) utilise SQLMap pour énumérer la base de données cible.

Commande d'attaque :

```
1 sqlmap -u "http://192.168.1.2/vulnerabilities/sqli/?id=1&Submit=Submit"
2 \
3     --cookie="PHPSESSID=<session_id>; security=low" \
4     --dbs --batch
```

Listing 16 – Attaque SQLMap depuis KALI

Remplacer `<session_id>` par le cookie PHP obtenu après connexion à DVWA. L'option `-dbs` énumère les bases de données ; `-batch` automatisé les confirmations. SQLMap génère des dizaines de requêtes HTTP malformées qui traversent le SIEM et déclenchent les règles Suricata.

Chaîne de détection :

1. SQLMap envoie des requêtes HTTP avec payloads SQL depuis KALI.
2. Les paquets traversent le SIEM — Suricata les inspecte.
3. La règle `ET WEB_SERVER SQL Injection Attempt` se déclenche.
4. L'alerte est écrite dans `eve.json`.
5. Filebeat sur SIEM expédie l'alerte vers Elasticsearch.
6. L'alerte est visible dans Kibana Discover sous `event.module: suricata`.
7. Simultanément, le log Apache sur WEB enregistre la requête.
8. Filebeat sur WEB expédie ce log vers Elasticsearch.
9. Les deux événements sont corrélables par `src_ip` et `@timestamp`.

5.1.2 Scénario 2 — Injection SQL via l’interface DVWA (navigateur)

Objectif : Comprendre les limites d’un IDS réseau face aux interactions locales.

Contexte : L’utilisateur entre manuellement un payload SQL (' OR '1'='1) dans le formulaire DVWA via un navigateur.

Observation : Aucune alerte Suricata n’est générée. L’interaction navigateur génère du trafic HTTP entre le poste client et le serveur, mais sans les signatures de scan automatisé que SQLMap produit. Cette limite illustre un principe fondamental : un IDS réseau détecte des patterns de trafic, pas des intentions humaines.

Enseignement : Pour une détection complète, un WAF (Web Application Firewall) ou un agent applicatif (type Wazuh) doit compléter l’IDS réseau.

5.2 Règles Suricata déclenchées

Les règles Emerging Threats suivantes ont été observées lors des tests :

Signature ID	Description	Catégorie
2006445	ET WEB_SERVER SQL Injection Attempt	Web Application Attack
2006446	ET WEB_SERVER SQLMap User-Agent	Web Application Attack
2001219	ET SCAN Potential SQL Injection	Attempted Information Leak

TABLE 5 – Règles Suricata déclenchées

5.3 Configuration des alertes Suricata

Les règles Suricata sont stockées dans `/var/lib/suricata/rules/` et mises à jour par `suricata-update`. Pour créer une règle personnalisée :

```
1 sudo nano /var/lib/suricata/rules/local.rules
```

Listing 17 – Création d’une règle Suricata personnalisée

```
1 alert tcp any any -> 192.168.1.0/24 any \
2   (msg:"Local Network Port Scan Detected"; \
3   flags:S; threshold:type threshold, track by_src, \
4   count 20, seconds 10; \
5   classtype:network-scan; sid:9000001; rev:1;)
```

Listing 18 – Exemple de règle personnalisée — détection d’un scan de port

```
1 sudo kill -USR2 $(pidof suricata)
```

Listing 19 – Rechargement des règles sans redémarrage

6 Analyse d’Incidents

6.1 Méthodologie d’analyse

L’analyse d’un incident dans cette architecture suit le cycle suivant :

1. **Détection** — une alerte apparaît dans Kibana (`event_type: alert`).
2. **Triage** — identifier l’IP source, la signature déclenchée, la sévérité.

3. **Contextualisation** — croiser avec les logs Apache pour confirmer l'attaque.
4. **Investigation** — analyser la timeline et les payloads.
5. **Qualification** — vrai positif, faux positif, ou incident avéré ?
6. **Remédiation** — blocage IP, correction applicative, mise à jour des règles.

6.2 Incident 1 — Attaque par injection SQL (SQLMap)

Champ	Valeur
Date/Heure	2026-06-02 14 :32 :10 UTC
IP Source	192.168.1.3 (KALI)
IP Destination	192.168.1.2 (WEB)
Port Destination	80 (HTTP)
Signature	ET WEB_SERVER SQL Injection Attempt
Sévérité	1 (Critique)
Outil détecté	sqlmap/1.7 (via User-Agent)
URL ciblée	/vulnerabilities/sqli/?id=1%27+AND+...
Réponse serveur	HTTP 200 (requête traitée)

TABLE 6 – Incident SQLMap

Analyse : L'attaque est un vrai positif confirmé. SQLMap a envoyé plusieurs centaines de requêtes en moins de 5 minutes. Le serveur Apache a répondu avec des codes 200, indiquant que les requêtes ont atteint l'application. Le User-Agent `sqlmap` est clairement identifiable dans les logs Apache. Suricata a déclenché des alertes dès la première requête anormale.

Remédiation recommandée :

- Bloquer l'IP 192.168.1.3 via iptables ou une règle IPS Suricata.
- Augmenter le niveau de sécurité DVWA à Medium ou High.
- Mettre en place un WAF devant Apache.

6.3 Incident 2 — Problème de configuration (faux incident)

Champ	Observation
Aucun événement dans Kibana malgré les attaques	
Cause réelle	Filebeat non configuré (fileset manquant)
Type	Faux négatif de configuration
Impact	Détection silencieuse — aucune alerte visible

TABLE 7 – Incident de configuration

Analyse : Ce type d'incident de configuration est particulièrement dangereux en production car le SIEM semble fonctionner sans générer d'erreurs, mais ne collecte rien. La validation end-to-end du pipeline de logs est une étape obligatoire lors de chaque déploiement ou modification.

Commandes de vérification du pipeline :

```

1 # Verifier le statut de Filebeat
2 sudo systemctl status filebeat
3
4 # Verifier les modules actifs
5 sudo filebeat modules list
6
7 # Tester la configuration
8 sudo filebeat test config -e
9
10 # Tester la connexion Elasticsearch
11 sudo filebeat test output

```

Listing 20 – Vérification complète du pipeline Filebeat

7 Dashboards Kibana / Zabbix

7.1 Kibana — Vue Discover

Le point d'entrée principal de l'analyse est l'interface **Discover** de Kibana, accessible à l'adresse <http://192.168.1.1:5601>. Elle permet d'explorer les documents indexés dans Elasticsearch en temps réel.

7.1.1 Index pattern

L'index pattern `filebeat-*` agrège tous les logs expédiés par les instances Filebeat. Il est créé via : Management → Stack Management → Index Patterns → Create index pattern

7.1.2 Filtres utilisés pour la corrélation

```

1 event.module: suricata AND source.ip: 192.168.1.3
2 event.module: apache AND source.ip: 192.168.1.3
3 event.module: suricata AND suricata.eve.alert.severity: 1

```

Listing 21 – Requêtes KQL pour la corrélation

7.1.3 Dashboards Kibana pré-intégrés via Filebeat

La commande `filebeat setup` installe automatiquement des dashboards Kibana pour chaque module :

Dashboard	Contenu
[Filebeat Apache] Access and error logs	Vue d'ensemble du trafic HTTP, codes de réponse, IPs les p
[Filebeat Suricata] Alert Overview	Alertes IDS dans le temps, top des signatures déclenchées, n
[Filebeat Suricata] Events Overview	Tous les événements Suricata (flows, DNS, HTTP, alertes) .

TABLE 8 – Dashboards Kibana pré-intégrés

7.2 Visualisations personnalisées recommandées

Pour aller au-delà des dashboards auto-générés, les visualisations suivantes sont recommandées dans Kibana Lens ou TSVB :

1. Histogramme des alertes Suricata dans le temps — permet de détecter des pics d'activité suspects.
2. Top 10 des signatures Suricata déclenchées — identifie les types d'attaques les plus fréquents.
3. Carte des IPs sources — géolocalisation des attaquants (pertinente en cas de trafic externe).
4. Taux de réponses HTTP 200/403/500 — un pic de 200 lors d'une attaque indique une exploitation réussie.
5. Nombre d'événements par event.module — surveillance de la santé du pipeline de collecte.

7.3 Zabbix — Monitoring Infrastructure

Zabbix est prévu pour surveiller les métriques système de chaque VM : utilisation CPU, mémoire disponible, espace disque, charge réseau sur les interfaces.

Statut : L'installation du serveur Zabbix a rencontré une erreur de paquets cassés liée à un dépôt incompatible avec la version Ubuntu. Le composant n'a pas pu être validé dans cet environnement. Les tableaux de bord Zabbix décrits ci-dessous correspondent à l'architecture cible.

Métriques Zabbix cibles par VM :

VM	Métrique	Seuil d'alerte
SIEM	CPU utilisation	> 80% pendant 5 min
SIEM	Mémoire libre	< 200 MB
SIEM	Interface réseau (in/out)	Pic anormal
WEB	Connexions Apache actives	> 500
WEB	Espace disque logs	< 1 GB libre
KALI	(monitoring optionnel)	—

TABLE 9 – Métriques Zabbix cibles

8 Logs Collectés

8.1 Récapitulatif des logs par source

Source	Fichier / Chemin	Contenu
Apache (WEB)	/var/log/apache2/access.log	méthode, URI, code HTTP, user-agent
Apache (WEB)	/var/log/apache2/error.log	PHP, erreurs serveur
Suricata (SIEM)	/var/log/suricata/eve.json	flows TCP/UDP, événements DNS/HTTP
Suricata (SIEM)	/var/log/suricata/fast.log	texte simplifié des alertes (diagnostic)
Filebeat (WEB)	/var/log/filebeat/filebeat.log	erreurs du shipper
Elasticsearch	Index filebeat-*	Tous les documents indexés (JSON)

TABLE 10 – Récapitulatif des logs par source

8.2 Exemple de log Apache annoté

```
1 192.168.1.3 - - [02/Jun/2026:14:32:10 +0000]
2 "GET /vulnerabilities/sqli/?id=1' AND 1=1-- &Submit=Submit HTTP/1.1"
3 200 4823
4 "http://192.168.1.2/vulnerabilities/sqli/"
5 "sqlmap/1.7 (#stable)"
6
7 # Champs :
8 # IP source      : 192.168.1.3 (KALI)
9 # Timestamp     : 02/Jun/2026 14:32:10
10 # Payload SQL   : id=1' AND 1=1-- (tentative de bypass auth)
11 # Code reponse  : 200 (requete traitee - potentielle exploitation)
12 # User-Agent    : sqlmap/1.7 (outil identifie)
```

Listing 22 – Log Apache — requête SQLMap décodée

8.3 Commandes de consultation des logs

```
1 # Logs Apache en direct
2 sudo tail -f /var/log/apache2/access.log
3
4 # Alertes Suricata en direct (format JSON)
5 sudo tail -f /var/log/suricata/eve.json | python3 -m json.tool
6
7 # Alertes Suricata format texte (lisibilit )
8 sudo tail -f /var/log/suricata/fast.log
9
10 # Logs Filebeat (debug)
11 sudo journalctl -u filebeat -f
12
13 # Verifier les documents dans Elasticsearch
14 curl http://localhost:9200/filebeat-*/_count?pretty
```

Listing 23 – Consultation des logs en temps réel

9 Leçons Apprises

1. **Un IDS sans routage est aveugle.** Suricata ne détecte que le trafic qui traverse son interface. La reconfiguration de la passerelle sur les VM distantes était la condition préalable absolue à toute détection. En environnement de production, cette visibilité est généralement assurée par un TAP réseau ou un port miroir sur le commutateur.
2. **La configuration d'un agent ne suffit pas ; les filesets doivent être activés.** Filebeat distingue l'activation d'un module et l'initialisation de ses filesets. Une configuration syntaxiquement correcte peut ne rien collecter si `filebeat setup` n'a pas été exécuté. Tout pipeline doit être validé end-to-end avant mise en production.
3. **Un service peut être actif et inaccessible.** Kibana lié à localhost est invisible sur le réseau malgré un statut `active (running)`. La vérification des liaisons réseau (`server.host`, `network.host`) est obligatoire sur tout service exposé.
4. **Suricata nécessite des règles avant le premier démarrage.** Un répertoire de règles vide provoque l'échec silencieux du service. `suricata-update` doit être exécuté en premier. En production, les mises à jour automatiques des règles sont recommandées via un cron job.
5. **Les attaques browser ne génèrent pas de signatures réseau détectables.** L'interaction manuelle avec un formulaire web produit du trafic HTTP normal sans marqueurs automatisés. La détection réseau requiert des outils qui génèrent du trafic à fort volume ou avec des User-Agents caractéristiques (SQLMap, Nikto, Metasploit).
6. **La corrélation multi-source est la valeur centrale d'un SIEM.** Un log Apache seul ou une alerte Suricata seule est insuffisante pour qualifier un incident. La puissance du SIEM réside dans la mise en relation de ces flux en temps réel.

10 Conclusion

Ce projet démontre concrètement la mise en place d'une infrastructure SOC fonctionnelle en environnement virtualisé. Partant d'un réseau plat sans visibilité, l'environnement a été progressivement transformé en un système capable de :

- **Détecter des attaques réelles** — les injections SQL automatisées de SQLMap déclenchent des règles Suricata et apparaissent sous forme d'alertes dans Kibana.
- **Centraliser les logs** — les journaux Apache de la VM WEB et les alertes Suricata de la VM SIEM convergent vers un index Elasticsearch unique.
- **Corréler les données applicatives et réseau** — une session Kibana permet de tracer une attaque depuis son IP source, à travers la requête HTTP, jusqu'à l'alerte IDS.
- **Documenter chaque obstacle** — les erreurs de configuration, de routage et de dépendances rencontrées reflètent fidèlement les défis réels d'un déploiement SOC, et leur résolution constitue en elle-même un apprentissage opérationnel.

Les composants déployés constituent une base solide et extensible. Les prochaines étapes naturelles incluent : l'intégration complète de Zabbix, la création de règles Suricata personnalisées, la mise en place d'alertes Kibana automatiques par e-mail, et l'ajout

d'un SOAR (Security Orchestration Automation and Response) comme Shuffle pour automatiser la réponse aux incidents.